

AgriScan.ai

Script de Présentation Orale

3 personnes

17 slides

~15–18 min

Distribution

Personne	Slides	Contenu
Personne 1	0–5	Cover, Problème, Solution, Fonctionnalités, Focus Diagnostic, Alertes
Personne 2	6–10	No/Low-Code, Architecture, Défis, Migration, Llama 70B
Personne 3	11–16	Impact, Auth, IoT, Admin, Perspectives, Conclusion

Personne 1 — Slides 0 à 5

Slide 0 — Cover

Bonjour à tous. Je suis ravi de vous présenter aujourd'hui notre projet d'étude : **AgriScan AI**.

AgriScan AI est une application mobile et web de **diagnostic intelligent des maladies des plantes**, développée dans le cadre de notre Bachelor 2025-2026, en réponse à une demande de la **Chambre d'Agriculture**.

L'idée est simple : permettre à un agriculteur de **photographier une plante suspecte** et d'obtenir instantanément un diagnostic — maladie, probabilité, urgence et traitement recommandé — le tout propulsé par l'intelligence artificielle.

Slide 1 — Problème

Avant de vous parler de la solution, posons le problème.

Aujourd'hui, quand un agriculteur voit une plante malade dans son champ, il ne peut pas toujours joindre un agronome rapidement. Le délai moyen d'un diagnostic est de **48 à 72 heures** — un délai critique pendant lequel la maladie peut se propager.

Conséquence : un traitement tardif peut entraîner jusqu'à **40% de perte de récolte**, ce qui est irréversible pour la saison.

Et pour couronner le tout, les alertes phytosanitaires sont dispersées entre différentes sources, sans **aucune visualisation géolocalisée** centralisée. L'agriculteur n'a pas une vue d'ensemble des menaces qui pèsent sur ses parcelles.

Slide 2 — Solution

C'est là qu'AgriScan AI intervient.

Une photo. Une analyse. Un traitement.

Notre application permet à l'utilisateur de :

- **Photographier** la plante suspecte directement depuis son mobile
- **Analyser** l'image via notre agent IA spécialisé en pathologie végétale
- **Traiter** grâce à une recommandation personnalisée

Le tout en **moins de 5 secondes**, sans avoir besoin d'un expert sur le terrain.

Slide 3 — Fonctionnalités

Concrètement, AgriScan AI s'articule autour de **5 fonctionnalités clés** :

- **Le Diagnostic IA** : le cœur de l'application — on prend une photo, l'IA identifie la maladie avec sa probabilité et propose un traitement
- **L'Assistant conversationnel** : un agent IA spécialisé avec lequel on peut discuter, dossier par dossier
- **Le Dashboard & Historique** : pour suivre ses diagnostics, ses statistiques et gérer son exploitation
- **Les Alertes géolocalisées** : une carte interactive avec les risques phytosanitaires
- **Le Profil & Exploitation** : gestion du compte et des paramètres personnels

Voici un aperçu de l'interface : la page d'accueil, la page de diagnostic IA et le profil utilisateur.

3 screenshots : [home](#), [diagnostic](#), [profil](#)

Slide 4 — Focus Diagnostic IA

Plongeons un peu plus dans la **feature principale** : le diagnostic par IA.

Contrairement à un simple chatbot, nous avons conçu un **véritable agent expert**. L'utilisateur envoie une photo, et le moteur Llama 70B — avec un system prompt spécialisé en pathologie végétale — analyse l'image et retourne un résultat structuré.

On obtient : **le nom de la maladie, un pourcentage de probabilité, un niveau d'urgence, et le traitement recommandé.**

Et si le résultat n'est pas assez précis, on peut dialoguer avec l'agent pour affiner le diagnostic.

Screenshot interface diagnostic : [Mildiou de la vigne — 87% — Urgence modérée](#)

Slide 5 — Focus Alertes & Carte

Pour finir ma partie, parlons des **alertes phytosanitaires**.

L'application intègre une **carte Google Maps interactive** qui permet à l'agriculteur de visualiser ses parcelles et les alertes en cours.

Chaque alerte est **classée par type et par sévérité** : critique, modérée, ou simple information. On peut voir d'un coup d'œil ce qui menace ses cultures avant même d'aller au champ.

L'application intègre aussi les **données météo**, des **données par zone** comme la température, l'humidité et le risque, et envoie des **notifications personnalisées** par parcelle avec un simple clic pour les détails.

Screenshot carte avec alertes classées par code couleur

C'est un outil de **vigilance temps réel** qui transforme la manière dont l'agriculteur surveille son exploitation.

Personne 2 — Slides 6 à 10

Slide 6 — Approche No/Low-Code

Merci. Je vais maintenant vous expliquer notre **approche de développement**.

Nous avons adopté une stratégie hybride : **No/Low-Code + agents IA**.

D'un côté, nous avons utilisé la **plateforme Emergent** — une solution no-code qui nous a permis d'accélérer le développement : infrastructure, base de données, authentification, stockage — tout était natif. Le **time-to-market a été divisé par 3**.

De l'autre côté, nous avons **superposé des agents IA spécialisés** : un system prompt pour la pathologie végétale, une mémoire de conversation par dossier diagnostic, et une analyse multimodale photo + texte.

Et troisièmement, face aux restrictions de la plateforme Emergent, nous avons dû opérer une **migration locale** dont je reparlerai. Le code a été exporté, adapté pour fonctionner en local avec FastAPI, Motor, MongoDB et les modèles Llama via Groq.

Ce n'est pas du no-code basique — c'est une **architecture hybride moderne**.

Slide 7 — Architecture

Regardons maintenant l'**architecture technique** dans son ensemble.

En haut, le **client** : une application React Native avec Expo SDK, compatible mobile et web, avec Google Maps intégré. La navigation est gérée par Expo Router et l'état global par Zustand.

Au milieu, le **serveur** : une API REST FastAPI en Python avec MongoDB pour la persistance des données. L'authentification est simplifiée — pas de JWT, on utilise directement l'identifiant utilisateur. Le serveur expose **~23 endpoints REST** couvrant l'auth, les diagnostics, le chat, les alertes et les statistiques. Tout ce code tourne en **local**, plus aucune dépendance au cloud Emergent.

Et enfin, la couche **IA** : nous utilisons l'API **Groq** (compatible OpenAI) avec deux modèles **Llama** — le **Llama 3.3-70B** pour les conversations et le **Llama 4 Scout 17B** pour l'analyse d'images. C'est une migration complète depuis l'ancien **GPT-5.2** d'Emergent.

C'est une architecture en **3 couches** — propre, maintenable, et 100% maîtrisée.

Slide 8 — Défis Surmontés

Bien sûr, ce projet n'a pas été un long fleuve tranquille. Voici les **3 défis majeurs** que nous avons dû surmonter.

Défi 1 — le cross-platform : faire tourner le même code sur iOS, Android et Web. Notre solution ? Expo avec détection Platform.OS, AsyncStorage côté web, SecureStore côté natif, et deux implémentations Google Maps différentes. Résultat : un seul codebase, trois plateformes.

Défi 2 — l'intégration IA sans casser l'UX : les appels à l'IA peuvent être lents. Notre solution ? Le **streaming des réponses** pour éviter les temps de chargement bloquants, et le parsing automatique des réponses structurées. Résultat : diagnostic en moins de 5 secondes.

Défi 3 — le plus critique : les **crédits Emergent épuisés**. La plateforme no-code imposait des limites strictes de crédits, rendant la facturation intenable. Nous avons dû exporter tout le code et le faire fonctionner en local. Résultat : une **migration réussie vers Groq et Llama** dont je vais vous parler.

Slide 9 — Migration Emergent → Local

Ce **pivot stratégique** a été un moment clé du projet.

Étape 1 : nous avons exporté l'intégralité du backend depuis la plateforme Emergent.

Étape 2 : nous avons adapté le code pour fonctionner en local avec **FastAPI, Motor et MongoDB** — plus aucune dépendance au cloud Emergent. Et côté IA, nous avons remplacé GPT-5.2 par **Llama 70B via Groq**.

Étape 3 : nous avons gagné notre **autonomie totale** — contrôle complet sur l'infrastructure, coûts maîtrisés, et une roadmap libre.

Ce qui aurait pu être un échec s'est transformé en opportunité : nous maîtrisons désormais **100% de notre stack technique**.

Slide 10 — Llama 70B via Groq

Pour terminer ma partie, parlons du **modèle d'inférence** que nous utilisons désormais.

Comparatif Avant/Après : GPT-5.2 (Emergent) → Llama 70B (Groq)

Avant, nous dépendions de **GPT-5.2** via la plateforme Emergent — un modèle puissant mais coûteux et limité par les crédits.

Maintenant, nous utilisons **Llama 70B** fourni par **Groq** — un modèle open source performant qui remplace GPT-5.2 pour **tous les usages** : à la fois la génération de code et les diagnostics agricoles. Pour l'analyse d'images, nous utilisons **Llama 4 Scout 17B** qui gère la vision.

Les avantages sont multiples :

- **Zéro coût d'API** : le modèle Groq est accessible gratuitement
- **Indépendance totale** : plus de crédits, plus de dépendance, plus de facturation imprévue
- **Backend local** : FastAPI autonome, IA via Groq (cloud gratuit)

Personne 3 — Slides 11 à 16

Slide 11 — Impact & Métriques

Merci. Alors, qu'est-ce que ce projet démontre concrètement ?

- **2 plateformes** : mobile et web, livrées simultanément
- **~23 endpoints API** : toute une API REST documentée
- **1 agent IA expert** en pathologie végétale
- **5 modules complets** : diagnostic, dashboard, alertes, profil, administration
- **Backend local, IA via Groq** : migration Emergent → FastAPI + Groq/Llama effective

Ce projet a été livré en **autonomie complète** — du design au déploiement, en passant par le mobile, le web, l'IA, le backend et le design system.

Slide 12 — Authentification & Profil

Regardons maintenant quelques fonctionnalités en détail, en commençant par la **gestion de compte**.

L'utilisateur peut s'inscrire et se connecter via email et mot de passe. Une fois connecté, il peut modifier son profil : photo, nom, email et mot de passe.

Screenshot page de profil utilisateur

Chaque utilisateur a un **rôle** — Admin ou Utilisateur — affiché avec un badge distinctif. L'authentification est simplifiée pour le MVP : pas de JWT, l'utilisateur est stocké en mémoire avec Zustand. C'est volontairement léger pour cette version.

Les routes privées sont protégées côté interface par la vérification du state Zustand.

Slide 13 — Capteurs IoT

Parlons maintenant de l'**infrastructure IoT** — un aspect clé de l'agriculture de précision.

Screenshot interface capteurs : 6 indicateurs (température, humidité, lumière, vent, pH, batterie) + connectivité LoRaWAN

Nous avons conçu et simulé des **capteurs** qui remontent en temps réel les données du terrain : température, humidité du sol, lumière PAR, vent, pH, batterie — le tout connecté via **LoRaWAN**.

Les données sont simulées côté frontend pour le moment, mais l'architecture IoT a été conçue pour fonctionner sur le terrain avec des capteurs réels. La connectivité affiche un signal de 4/5 avec une transmission toutes les 3 minutes.

C'est une **architecture IoT complète**, prête à être déployée physiquement.

Slide 14 — Administration

Côté **back-office**, l'administration est complète.

Screenshot panneau d'administration : liste utilisateurs + statut modèle IA

La gestion des utilisateurs permet de voir tous les comptes, leurs rôles, et de les administrer directement. Le panneau affiche aussi le **statut du modèle IA** — Llama 70B via Groq — toujours actif, sans limite de crédits.

C'est un outil de pilotage complet pour les administrateurs de la plateforme.

Slide 15 — Perspectives & Évolution

Regardons maintenant les **prochaines étapes** pour AgriScan AI.

- 1. Capteurs Réels** — l'infrastructure IoT est conçue et simulée. La prochaine étape est de déployer des capteurs LoRaWAN sur une parcelle test pour remonter de vraies données de température, humidité et luminosité.
- 2. Mode Hors-Ligne** — nous voulons implémenter un cache local qui permet de prendre des photos et de poser des diagnostics même sans connexion, avec synchronisation automatique au retour du réseau.
- 3. Notifications Push** — envoyer des alertes en temps réel via expo-notifications quand un risque est détecté sur une parcelle.
- 4. Tests Terrain** — valider l'application sur des cas réels en partenariat avec la Chambre d'Agriculture, pour mesurer la précision des diagnostics sur le terrain.

Ce projet est une **base solide**, et ces évolutions le transformeront en un outil professionnel complet.

Slide 16 — Conclusion / Remerciements

Pour conclure, AgriScan AI, c'est :

- 1. Du No/Low-Code** : nous avons utilisé la plateforme Emergent pour un prototypage rapide.
- 2. De l'Intelligence Artificielle** : un agent IA spécialisé en pathologie végétale, propulsé par Llama 70B via Groq.
- 3. Une migration réussie** : du cloud Emergent (GPT-5.2) vers une architecture locale (FastAPI backend) + IA via Groq (Llama).

Un grand merci à la **Chambre d'Agriculture** pour ce cahier des charges, à notre établissement pour l'encadrement, et à vous pour votre attention.

Des questions ?

Récapitulatif du temps

Personne	Slides	Durée estimée
Personne 1	0–5 (6 slides)	~5–6 min
Personne 2	6–10 (5 slides)	~5–6 min
Personne 3	11–16 (6 slides)	~5–6 min
Total	17 slides	~15–18 min

Conseils de présentation

- **Transitions** : quand une personne finit, dire "Je passe la parole à..." pour fluidité
- **Slides 8-10** (Défis → Migration → Llama) : à enchaîner sans pause — c'est l'histoire centrale du projet
- **Démo** : prévoir d'ouvrir l'application locale sur mobile ou navigateur pendant les questions
- **Ton** : rester naturel, ne pas lire mot à mot — le script est une base, adaptez le langage à l'oral